Reto 6 – Automatización de Pruebas

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. Hasta ahora, se ha construido un sistema distribuido con:

- Servicios de negocio con persistencia y comunicación REST (Retos 1 y 2)
- Comunicación asincrónica basada en eventos (Reto 4)
- Seguridad centralizada con JWT, RBAC y ciclo de vida de cuentas (Reto 5)

A lo largo de los retos anteriores, cada equipo ha verificado el funcionamiento del sistema ejecutando peticiones manualmente con herramientas como `curl` o Postman, siguiendo los flujos de prueba sugeridos paso a paso. Sin embargo, estas verificaciones no son repetibles, no están documentadas de forma ejecutable, y cada vez que se modifica un servicio, hay que repetir todo el proceso desde cero.

Hay además un problema más silencioso: **el ecosistema no tiene una sola prueba automatizada**. La lógica de negocio más delicada del sistema —la validación de solapamiento de vacaciones, la decisión de no reactivar la cuenta de un empleado retirado, la unicidad del número de empleado— no está cubierta por nada. Cualquier refactor puede romperla sin que nadie se entere hasta que un humano lo pruebe a mano.

En este reto se construye la **estrategia de pruebas completa** del ecosistema: pruebas unitarias y de integración por microservicio, y pruebas funcionales de extremo a extremo con **BDD (Behavior-Driven Development)**. Todas deben producir **reportes en formato estándar**, porque en el Reto 7 el pipeline de Jenkins los va a consumir.

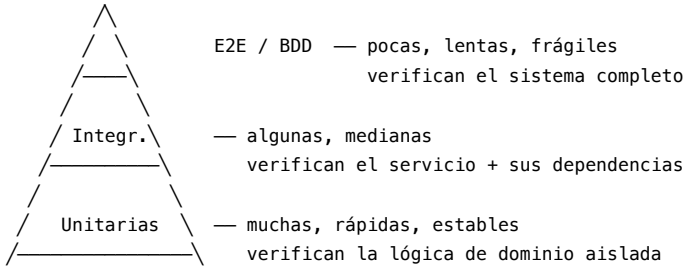
Objetivo

Implementar la estrategia de pruebas automatizadas del ecosistema en sus tres niveles: **unitarias** (lógica de dominio aislada), **de integración** (el servicio contra sus dependencias reales) y **funcionales de extremo a extremo** (BDD con Gherkin contra el sistema completo desplegado). Todas las suites deben ejecutarse por comando y **generar reportes en formatos estándar** listos para ser publicados por un servidor de integración continua.

Introducción

La pirámide de pruebas

No todas las pruebas cuestan lo mismo ni detectan lo mismo. La **pirámide de pruebas** es la heurística clásica para decidir cuántas escribir de cada tipo:



Nivel	Qué prueba	Qué se sustituye	Velocidad	Cuándo falla te dice
Unitaria	Una clase o función de lógica de negocio	Todo lo externo (BD, HTTP, broker)	milisegundos	*Exactamente* qué regla se rompió
Integración	El servicio contra su BD y/o su broker reales	Solo los otros microservicios	segundos	Que la pieza no encaja con su infraestructura

Nivel	Qué prueba	Qué se sustituye	Velocidad	Cuándo falla te dice
E2E / BDD	El flujo de negocio completo, varios servicios	Nada	minutos	Que el sistema, como un todo, no cumple

> **Por qué importa el orden.** Si el 100 % de sus pruebas son E2E, cada fallo le dice "*algo se rompió en algún lado*" y usted tarda una hora en encontrar qué. Si tiene una base sólida de pruebas unitarias, el mismo fallo aparece como " `Vacaciones.validarSolapamiento` *devolvió* `false` *cuando debía devolver* `true` ", y lo corrige en cinco minutos.

En este reto se construyen los tres niveles: el **Punto 1** arma la base de la pirámide, los **Puntos 2 a 5** construyen la punta, y el **Punto 6** hace que todo produzca reportes publicables.

¿Qué es BDD?

Behavior-Driven Development (BDD) es una metodología que extiende el desarrollo guiado por pruebas (TDD) al enfocar las pruebas en el **comportamiento** del sistema desde la perspectiva del usuario, no en la implementación técnica.

Aspecto	Pruebas Tradicionales	BDD
Perspectiva	Técnica (funciones, clases, módulos)	Negocio (flujos, comportamiento, reglas)
Lenguaje	Código del lenguaje de programación	Lenguaje natural estructurado (Gherkin)
Audiencia	Desarrolladores	Desarrolladores, QA, stakeholders
Documentación	Se desactualiza fácilmente	**Documentación viva**: si la prueba pasa, el comportamiento documentado es real
Pregunta clave	¿Funciona esta función?	¿El sistema se comporta correctamente?

El ciclo BDD sigue estas fases:

1. **Descubrir** — Identificar escenarios a partir de las historias de usuario
2. **Formular** — Escribir los escenarios en **Gherkin** (lenguaje natural estructurado)
3. **Automatizar** — Implementar los **step definitions** (código que ejecuta cada paso)
4. **Ejecutar** — Correr las pruebas; si fallan, desarrollar la funcionalidad
5. **Iterar** — Repetir para cada nueva funcionalidad o cambio

Sintaxis Gherkin

Gherkin es el lenguaje utilizado para escribir escenarios BDD. Se estructura con palabras clave que describen el contexto, la acción y el resultado esperado.

Palabras clave

Gherkin soporta múltiples idiomas de forma nativa. Para este reto, se utilizarán las palabras clave en español (activando `# language: es` en la primera línea del archivo `.feature`):

Inglés	Español
`Feature`	`Característica`
`Scenario`	`Escenario`
`Scenario Outline`	`Esquema del escenario`
`Given`	`Dado` / `Dada` / `Dados` / `Dadas`
`When`	`Cuando`
`Then`	`Entonces`
`And`	`Y`
`But`	`Pero`
`Examples`	`Ejemplos`

Inglés	Español
`Background`	`Antecedentes`

Estructura de un archivo `.feature`

- **Característica (Feature):** Funcionalidad o historia de usuario que se está probando.
- **Antecedentes (Background):** Precondiciones comunes a todos los escenarios del archivo.
- **Escenario (Scenario):** Un criterio de aceptación específico, con pasos `Dado/Cuando/Entonces`.
- **Esquema del escenario (Scenario Outline):** Escenario que se repite con diferentes datos, definidos en una tabla de `Ejemplos`.

Ejemplo

```
# language: es

Característica: Registro de empleados
  Como administrador del sistema
  Quiero registrar nuevos empleados
  Para iniciar su proceso de onboarding

  Antecedentes:
    Dado que existe un departamento "IT" con nombre "Tecnología"
    Y que estoy autenticado como "ADMIN"

  Escenario: Registro exitoso de un empleado
    Cuando registro un empleado con los siguientes datos:
      | id    | nombre     | email              | departamentoId |
      | E001  | Juan Pérez | juan@empresa.com   | IT             |
    Entonces la respuesta debe tener código 201
    Y el cuerpo debe contener el nombre "Juan Pérez"
```

Framework Cucumber

Cucumber es la herramienta que conecta los archivos `.feature` (Gherkin) con código ejecutable:

```
📄 .feature (Gherkin)  →  🥒 Cucumber (Runner)  →  📊 Reportes
⚙️ Step Definitions (Código)  ↗
```

- Los **archivos `.feature`** describen **qué** debe hacer el sistema (lenguaje natural).
- Los **step definitions** describen **cómo** se ejecuta cada paso (código).
- **Cucumber** lee los `.feature`, ejecuta los step definitions asociados y genera reportes.

Cada línea de un escenario se vincula con un método anotado en código. Por ejemplo, la línea `Dado que estoy autenticado como "ADMIN"` se asocia con:

```
@Dado("que estoy autenticado como {string}")
public void autenticarComo(String rol) {
    // Código que realiza la autenticación
}
```

Clientes HTTP para Step Definitions

Los step definitions deben ejecutar **peticiones HTTP reales** contra el sistema desplegado. Algunas librerías disponibles por lenguaje:

Lenguaje	Librerías HTTP
Java	`java.net.http` (Java 11+), Rest Assured (recomendado para pruebas), OkHttp, Feign
Python	requests, httpx
Node.js	axios, node-fetch

Lenguaje	Librerías HTTP
Go	net/http

Ejemplo con **Rest Assured** (Java):

```
@Cuando("consulto la lista de empleados")
public void consultarEmpleados() {
    response = given()
        .header("Authorization", "Bearer " + token)
        .when()
        .get(BASE_URL + "/empleados");
}

@Entonces("la respuesta debe tener código {int}")
public void verificarCodigo(int codigo) {
    response.then().statusCode(codigo);
}
```

Manejo de la Asincronía: Polling

En un sistema basado en eventos, la **eventual consistencia** es un desafío: cuando se crea un empleado, los otros servicios procesan el evento de forma asincrónica. No es seguro consultar inmediatamente el resultado.

Estrategia: Polling con reintentos. Consultar repetidamente hasta que la condición se cumpla o expire un timeout.

```
función esperarHastaQue(condicion, maxIntentos, intervaloMs):
    para i en 1..maxIntentos:
        intentar:
            si condicion() == verdadero:
                retornar éxito
        capturar error:
            // Todavía no está listo, reintentar
        esperar(intervaloMs)
    lanzar TimeoutException("Condición no cumplida después de " + maxIntentos + " intentos")
```

Parámetro	Valor sugerido	Justificación
Máximo de intentos	10-15	Suficiente para la mayoría de procesamiento de eventos
Intervalo entre intentos	1-2 segundos	Balance entre velocidad y carga
Timeout total	~30 segundos	Límite razonable para procesamiento asincrónico

> **Importante:** No utilice `sleep` fijo (ej. `Thread.sleep(5000)`). El polling es superior porque la prueba termina más rápido si el evento se procesa rápidamente, y tolera variaciones de tiempo sin fallar de forma intermitente.

> **Nota:** Para profundizar en estos conceptos, revise el material de clase sobre BDD.

Requisitos Generales

- Implementar **pruebas unitarias** de la lógica de dominio en al menos **dos microservicios**, con cobertura mínima del **70 %** en al menos uno de ellos.
- Implementar **pruebas de integración** que verifiquen el servicio contra su base de datos real.
- Utilizar un framework BDD compatible con Cucumber (o equivalente) según el lenguaje elegido.
- Escribir los escenarios en archivos `.feature` utilizando la sintaxis **Gherkin en español** (`# language: es`).
- Implementar las **definiciones de pasos (step definitions)** que ejecuten peticiones HTTP reales contra el sistema desplegado.
- Cada suite debe ejecutarse con un **único comando** y **generar su reporte en un formato estándar**, no solo imprimir en consola.
- Los reportes deben quedar en rutas conocidas y documentadas: el pipeline del **Reto 7** los va a publicar automáticamente.

Retos

Los siguientes seis puntos están organizados en orden de complejidad creciente. Se recomienda abordarlos en secuencia, ya que cada punto construye sobre las habilidades y el código del anterior.

Punto 1: Pruebas unitarias, de integración y cobertura (1.5 pt)

Lo que se entrega

La base de la pirámide. Este punto no toca el sistema desplegado: prueba **el código de cada microservicio por dentro**.

Qué es una prueba unitaria en este contexto

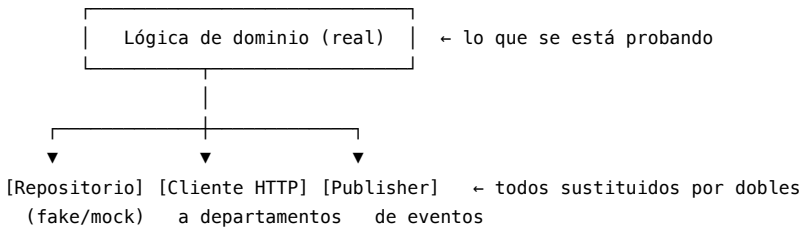
Una prueba unitaria verifica **una regla de negocio, aislada de toda infraestructura**. No levanta Docker, no abre conexiones, no publica eventos. Sustituye esas dependencias por **test doubles** (mocks, stubs, fakes) y se ejecuta en milisegundos.

El ecosistema ya tiene lógica de dominio que merece esta protección:

Servicio	Regla de negocio a probar unitariamente
`empleados-service`	Unicidad de `email` y `numeroEmpleado`; transición de estado a `RETIRADO` con `fechaRetiro`; rechazo de transiciones inválidas
`vacaciones-service`	`fechaFin > fechaInicio`; rechazo de fechas pasadas; **detección de solapamiento de períodos**; cálculo de días
`auth-service`	Validación y expiración del token de activación; hash de contraseña; **la cuenta de un empleado retirado no se reactiva al llegar `vacaciones.finalizadas`**
`perfiles-service`	Construcción del perfil por defecto a partir del evento; archivado

> **El caso del solapamiento de vacaciones es el ejemplo perfecto de prueba unitaria.** Es lógica pura sobre dos rangos de fechas, tiene muchos casos borde (el nuevo período empieza dentro del viejo, lo contiene, lo toca por un día, es idéntico…) y probarlo por HTTP sería absurdamente lento. Escríbalo como una tabla de casos.

Test doubles: qué sustituir



En el `empleados-service`, la prueba unitaria de "no se puede registrar un empleado con email duplicado" **no debe** consultar PostgreSQL ni llamar al servicio de departamentos: se le entrega un repositorio falso que ya contiene ese email y un cliente de departamentos que siempre responde que el departamento existe.

Qué es una prueba de integración

Verifica que el servicio funcione **contra su infraestructura real** —típicamente su base de datos—, pero **sin levantar los demás microservicios**. Es donde se detectan los errores que las unitarias no ven: un mapeo de entidad mal hecho, una restricción de unicidad que existe en el código pero no en el schema, una consulta con sintaxis inválida.

Estrategias, de mejor a más simple:

Estrategia	Cómo funciona	Ventaja / Desventaja
Testcontainers	Levanta un contenedor real de PostgreSQL/Mongo durante la prueba y lo destruye al terminar	Idéntico a producción / requiere Docker disponible en el entorno de pruebas

Estrategia	Cómo funciona	Ventaja / Desventaja
Base de datos embebida	H2, SQLite, Mongo en memoria	Rápida y sin dependencias / no es el mismo motor: puede pasar la prueba y fallar en producción
Instancia dedicada de pruebas	Un servicio extra en el Compose, solo para pruebas	Simple / hay que limpiar el estado entre ejecuciones

Frameworks y generación de cobertura por lenguaje

Lenguaje	Framework de pruebas	Mocking	Comando con cobertura	Reporte gen
Java (Maven)	JUnit 5	Mockito	`mvn verify`	`target/site/jacoco/jacoco.`
Java (Gradle)	JUnit 5	Mockito	`gradle test jacocoTestReport`	`build/reports/jacoco/.../j`
Python	pytest	`unittest.mock` , pytest-mock	`pytest --cov --cov-report=xml --junitxml=report.xml`	`coverage.xml + report.xml`
Node.js	Jest / Mocha	Jest mocks, sinon	`jest --coverage --reporters=default --reporters=jest-junit`	`coverage/lcov.info + juni`
Go	`testing`	gomock, testify	`go test -coverprofile=coverage.out ./...`	`coverage.out`
.NET	xUnit / NUnit	Moq	`dotnet test --collect:"XPlat Code Coverage" --logger trx`	`coverage.cobertura.xml +`

> **Los dos reportes son distintos y ambos hacen falta.** El de **resultados** (JUnit XML, `.trx`) dice *qué pruebas pasaron y cuáles fallaron*. El de **cobertura** (JaCoCo XML, lcov, Cobertura) dice *qué porcentaje del código ejecutaron*. En el Reto 7, Jenkins publica el primero y SonarQube consume el segundo.

Lo que debe hacer

1. **Escribir pruebas unitarias** de la lógica de dominio en **al menos dos microservicios en lenguajes diferentes**. Como mínimo debe cubrir:
   - Las cuatro validaciones del `vacaciones-service` , incluido el solapamiento con sus casos borde.
   - Las validaciones de unicidad y la transición a `RETIRADO` del `empleados-service` .
   - Al menos un caso de **error**, no solo el camino feliz.
2. **Usar test doubles** para aislar la lógica de la base de datos, del cliente HTTP a departamentos y del publicador de eventos. Ninguna prueba unitaria debe requerir Docker.
3. **Escribir pruebas de integración** de al menos un microservicio contra su base de datos real, verificando que la persistencia y las restricciones del schema funcionan.
4. **Configurar la generación de reportes** en **ambos** formatos (resultados y cobertura), en las rutas estándar de su herramienta. Documentar en el README la ruta exacta de cada reporte por servicio: el pipeline del Reto 7 va a necesitarlas.
5. **Alcanzar cobertura ≥ 70 %** en al menos uno de los microservicios, medida sobre la lógica de negocio. Verificarlo abriendo el reporte de cobertura.
6. **Verificar que las pruebas fallan cuando deben**: rompa deliberadamente una regla de negocio (por ejemplo, invierta la comparación de fechas) y confirme que una prueba unitaria lo detecta y señala exactamente cuál.

Decisión técnica requerida

- **Qué NO probar unitariamente.** Escribir pruebas de getters, setters o de código sin lógica infla la cobertura sin aportar seguridad. ¿Qué decidió excluir y por qué? ¿Configuró exclusiones en la herramienta de cobertura?
- **Estrategia de integración:** ¿Testcontainers, base embebida o instancia dedicada? Justifique el intercambio entre fidelidad y velocidad.
- **La trampa del 70 %:** la cobertura mide líneas ejecutadas, **no** que las aserciones sean correctas. Una suite puede tener 90 % de cobertura y no verificar nada. ¿Cómo se aseguró de que sus pruebas realmente comprueban comportamiento?

Punto 2: Configuración del proyecto de pruebas BDD (0.5 pt)

Lo que se entrega

La siguiente tabla resume los frameworks BDD y librerías HTTP disponibles por lenguaje:

Lenguaje	Framework BDD	Librería HTTP	Comando de ejecución
Java	Cucumber-JVM	Rest Assured / OkHttp / java.net.http	`mvn test`
Python	Behave / pytest-bdd	requests / httpx	`behave` o `pytest`
Node.js	Cucumber.js	axios / node-fetch	`npx cucumber-js`
Go	Godog	net/http	`godog`

> **Referencia:** Tutorial de Cucumber – 10 minutos

Lo que debe hacer

1. **Crear un proyecto de pruebas independiente** (`e2e-tests/` o nombre equivalente) separado del código de los microservicios, con la siguiente estructura base:

   ```
   e2e-tests/
   ├── features/              ← archivos .feature (Gherkin)
   ├── step_definitions/      ← implementación de los pasos
   ├── support/               ← contexto compartido, hooks, utilidades
   └── [archivo de proyecto]  ← pom.xml, package.json, requirements.txt, etc.
   ```

2. **Elegir y configurar** el framework BDD y la librería HTTP que utilizará.
3. **Diseñar el mecanismo de contexto compartido** entre pasos (también llamado "World" en algunos frameworks). Este contexto debe permitir:
   - Almacenar el token de autenticación actual
   - Almacenar la última respuesta HTTP recibida
   - Acceder a la URL base del sistema mediante variable de entorno
4. **Configurar variables de entorno** para que las pruebas no dependan de valores hardcodeados:
   - `BASE_URL` — URL base del sistema (ej. `http://localhost:8080`)
   - Credenciales de los usuarios de prueba (ADMIN, USER)
5. **Verificar la configuración** creando un escenario mínimo de "humo" que confirme que el sistema está operativo:

   ```
   # language: es

   Característica: Verificación del sistema
     Escenario: El sistema responde correctamente
       Dado que el sistema está desplegado y operativo
       Entonces la respuesta debe tener código 200
   ```

   Implemente el step definition correspondiente y ejecute la prueba.

Decisión técnica requerida

Usted debe decidir cómo estructurar el contexto compartido. Algunas preguntas que debe resolver:

- ¿Cómo se almacena y reutiliza el token entre pasos `Dado` y `Cuando` ?
- ¿Se crea una nueva instancia de contexto por escenario o se reutiliza?
- ¿Cómo se gestiona la autenticación en los `Antecedentes` ?

Punto 3: Escenarios de seguridad y control de acceso (0.5 pt)

Lo que se entrega

Un escenario de ejemplo que valida el acceso sin token:

```gherkin
# language: es

Característica: Seguridad y control de acceso
  Como sistema de autenticación
  Quiero controlar el acceso a los recursos
  Para garantizar que solo los usuarios autorizados realicen operaciones

  Escenario: Acceso denegado sin token de autenticación
    Cuando consulto la lista de empleados sin token de autenticación
    Entonces la respuesta debe tener código 401
```

Lo que debe hacer

1. **Partir del escenario de ejemplo** proporcionado e implementar su step definition.
2. **Escribir al menos 3 escenarios adicionales** que validen las reglas de seguridad RBAC implementadas en el Reto 5. Los escenarios deben cubrir, como mínimo:
   - Acceso con token inválido o malformado
   - Usuario con rol `USER` puede consultar recursos pero **no puede** modificarlos (crear, eliminar)
   - Usuario con rol `ADMIN` puede realizar todas las operaciones
3. **Implementar todos los step definitions** necesarios.
4. **Usar `Antecedentes`** para factorizar las precondiciones comunes entre escenarios del mismo archivo `.feature`.

Por qué este punto va primero que onboarding/offboarding

Los escenarios de seguridad son **100% sincrónicos** (request → response inmediata). Esto le permite enfocarse en aprender la mecánica de Gherkin + step definitions sin la complejidad adicional de la asincronía. Una vez dominados estos escenarios, los siguientes puntos agregan esa capa.

Punto 4: Escenarios de onboarding con verificación asincrónica (1.0 pt)

Lo que se entrega

El **concepto de polling** como estrategia para manejar la eventual consistencia en un sistema de eventos:

Cuando se crea un empleado, eventos asincrónicos disparan acciones en otros servicios (creación de credenciales, envío de notificaciones). No es posible verificar inmediatamente el resultado del evento. La estrategia es **consultar repetidamente** hasta que la condición se cumpla o se agote un timeout.

```
función esperarHastaQue(condicion, maxIntentos, intervaloMs):
    para i en 1..maxIntentos:
        intentar:
            si condicion() == verdadero:
                retornar éxito
        capturar error:
            // Todavía no está listo, reintentar
        esperar(intervaloMs)
    lanzar TimeoutException("Condición no cumplida después de " + maxIntentos + " intentos")
```

Convención Gherkin: Cuando un paso requiere polling, se usa la palabra **"eventualmente"** para indicar que no se espera resultado inmediato:

```gherkin
Entonces eventualmente el servicio de autenticación debe haber creado un usuario para "juan@empresa.com"
```

Lo que debe hacer

1. **Implementar la función de polling** en su lenguaje elegido, basándose en el pseudocódigo proporcionado.
2. **Definir los parámetros de polling** (máximo de intentos, intervalo, timeout total) y **justificar su elección** en el README.
3. **Escribir los escenarios de onboarding** que cubran:
   - Registro exitoso de un empleado → verificar que se generaron credenciales (evento asincrónico)
   - Registro exitoso → verificar que se generó una notificación (evento asincrónico)

- El nuevo empleado puede establecer su contraseña y hacer login exitosamente
- Registro con datos inválidos (departamento inexistente, campos faltantes) → error

4. **Implementar todos los step definitions**, usando polling para los pasos que verifican resultados asincrónicos.
5. **Garantizar independencia entre escenarios:** Cada escenario debe poder ejecutarse de forma aislada, sin depender del resultado de otro.

Decisión técnica requerida

- **Aislamiento de datos:** ¿Cómo garantiza que un escenario no deje "basura" que afecte a otro? ¿Usa IDs únicos generados? ¿Limpia el estado en hooks? ¿Usa `Antecedentes` para crear el estado necesario?
- **Calibración del polling:** ¿Qué timeout es razonable para su sistema? ¿Cómo lo calibró?

> **Importante:** No se aceptan `Thread.sleep()` o `time.sleep()` fijos. El polling es superior porque la prueba termina más rápido si el evento se procesa rápidamente, y tolera variaciones de tiempo sin fallar de forma intermitente.

Punto 5: Escenarios de vacaciones y offboarding (1.0 pt)

Lo que se entrega

Nada. Usted diseña estos escenarios con base en lo aprendido en los puntos 3 y 4.

Lo que debe hacer

1. **Diseñar y escribir los escenarios de vacaciones**, cubriendo al menos:
   - Programación exitosa de un período → verificar notificación asincrónica de confirmación
   - Rechazo por fechas incoherentes (`fechaFin` anterior a `fechaInicio`)
   - Rechazo por solapamiento con un período existente
   - **Ciclo completo:** las vacaciones inician → el empleado **no puede** hacer login → las vacaciones terminan → el empleado **vuelve a** poder hacer login
2. **Diseñar y escribir los escenarios de offboarding**, cubriendo al menos:
   - Desvinculación completa: retirar empleado → verificar notificación asincrónica de desvinculación
   - El empleado retirado **no puede** hacer login
   - La recuperación de contraseña **falla** para un empleado retirado
   - El empleado retirado **aparece** en la consulta de auditoría `GET /empleados?estado=RETIRADO`
3. **Implementar los step definitions**, reutilizando pasos ya creados en los puntos anteriores cuando sea posible.
4. **Manejar las precondiciones complejas:** los escenarios necesitan un empleado con credenciales activas y configuradas. Esto implica que en los `Antecedentes` o en un hook de setup se debe recrear ese estado desde cero.

El escenario más difícil del reto

El ciclo completo de vacaciones es el escenario más exigente de toda la suite, porque combina **tres tipos de espera distintos**: la respuesta síncrona del `POST` , la propagación del evento por el broker, y el **paso del tiempo** hasta que el scheduler dispara.

```
# language: es

Característica: Suspensión de cuenta durante vacaciones

  Antecedentes:
    Dado que existe un empleado activo con credenciales establecidas
    Y que estoy autenticado como "ADMIN"

  Escenario: La cuenta se suspende al iniciar las vacaciones y se reactiva al terminar
    Cuando programo un período de vacaciones que inicia de inmediato
    Entonces eventualmente el empleado no puede iniciar sesión
    Cuando el período de vacaciones finaliza
    Entonces eventualmente el empleado puede iniciar sesión nuevamente
```

Los pasos `Cuando programo un período que inicia de inmediato` y `Cuando el período finaliza` son los que debe resolver usted: el scheduler del Reto 5 corre a intervalos, así que la prueba debe provocar o esperar el disparo sin volverse lenta ni intermitente.

Decisión técnica requerida

- **Setup complejo:** ¿Cómo prepara el estado inicial (empleado registrado + credenciales activas) sin depender de otros escenarios? Algunas opciones:
  - Usar `Antecedentes` que invoquen los endpoints de creación
  - Usar hooks `Before` que configuren los datos necesarios
  - Usar datos precargados en la base de datos
- **Control del tiempo:** ¿cómo hace que un período de vacaciones inicie y termine dentro de una prueba, sin esperar días? ¿Usa fechas con hora, baja el intervalo del scheduler por variable de entorno, o llama al endpoint de disparo manual? **Justifique el enfoque y explique cómo evita que la prueba quede acoplada a la frecuencia del scheduler.**
- **Polling sobre una condición negativa:** verificar que el empleado "eventualmente **no** puede entrar" es distinto a verificar que algo aparece. ¿Cómo distingue "*todavía no se procesó el evento*" de "*el sistema está mal y nunca lo va a procesar*"?
- **Reutilización de pasos:** ¿Qué pasos de los puntos anteriores puede reutilizar directamente? ¿Cuáles necesitan ser parametrizados?

Punto 6: Reportes, reproducibilidad y documentación (0.5 pt)

Lo que se entrega

Una prueba que solo imprime en consola sirve para el desarrollador que la ejecuta y para nadie más. Para que un servidor de CI pueda decir "*el build falló porque el escenario X no pasó*", la suite debe **escribir sus resultados en un archivo con un formato que las herramientas sepan leer**.

Los dos formatos que debe producir la suite BDD

Formato	Para qué sirve	Quién lo consume
JUnit XML	Resultado máquina-legible: cuántos escenarios pasaron, fallaron o se omitieron, con el mensaje de error de cada fallo	Jenkins (paso `junit`), GitLab CI, GitHub Actions
Cucumber JSON / HTML	Reporte navegable con cada `Característica`, `Escenario` y paso, con su duración y estado	El plugin *Cucumber Reports* o *HTML Publisher* de Jenkins; también los humanos

> **Por qué los dos.** El JUnit XML es el que hace que el build **falle** y el que alimenta las gráficas de tendencia de pruebas. El HTML es el que un miembro del equipo abre para entender *qué* se rompió, en lenguaje de negocio. En el Reto 7 se publican ambos.

Cómo generarlos por framework

Framework	Configuración
Cucumber-JVM (Java)	`@CucumberOptions(plugin = {"pretty", "html:target/cucumber-report.html", "json:target/cucumber.json", "junit:target/cu` o la propiedad `cucumber.plugin` en `junit-platform.properties`
Behave (Python)	`behave --junit --junit-directory reports/ -f json -o reports/cucumber.json`
pytest-bdd (Python)	`pytest --junitxml=reports/junit.xml --cucumberjson=reports/cucumber.json`
Cucumber.js (Node)	`npx cucumber-js --format html:reports/cucumber.html --format json:reports/cucumber.json --format junit:reports/junit.x`
Godog (Go)	`godog run --format junit > reports/junit.xml` (y `--format cucumber` para el JSON)

Convención de rutas

Defina una convención y respétela en todos los servicios, porque el pipeline del Reto 7 va a apuntar a estas rutas:

```
<servicio>/
├── reports/
│   ├── junit.xml          ← resultados (unitarias / integración)
│   └── coverage.xml       ← cobertura
e2e-tests/
└── reports/
    ├── junit.xml          ← resultados de los escenarios BDD
    ├── cucumber.json      ← reporte estructurado
    └── cucumber.html      ← reporte navegable
```

Lo que debe hacer

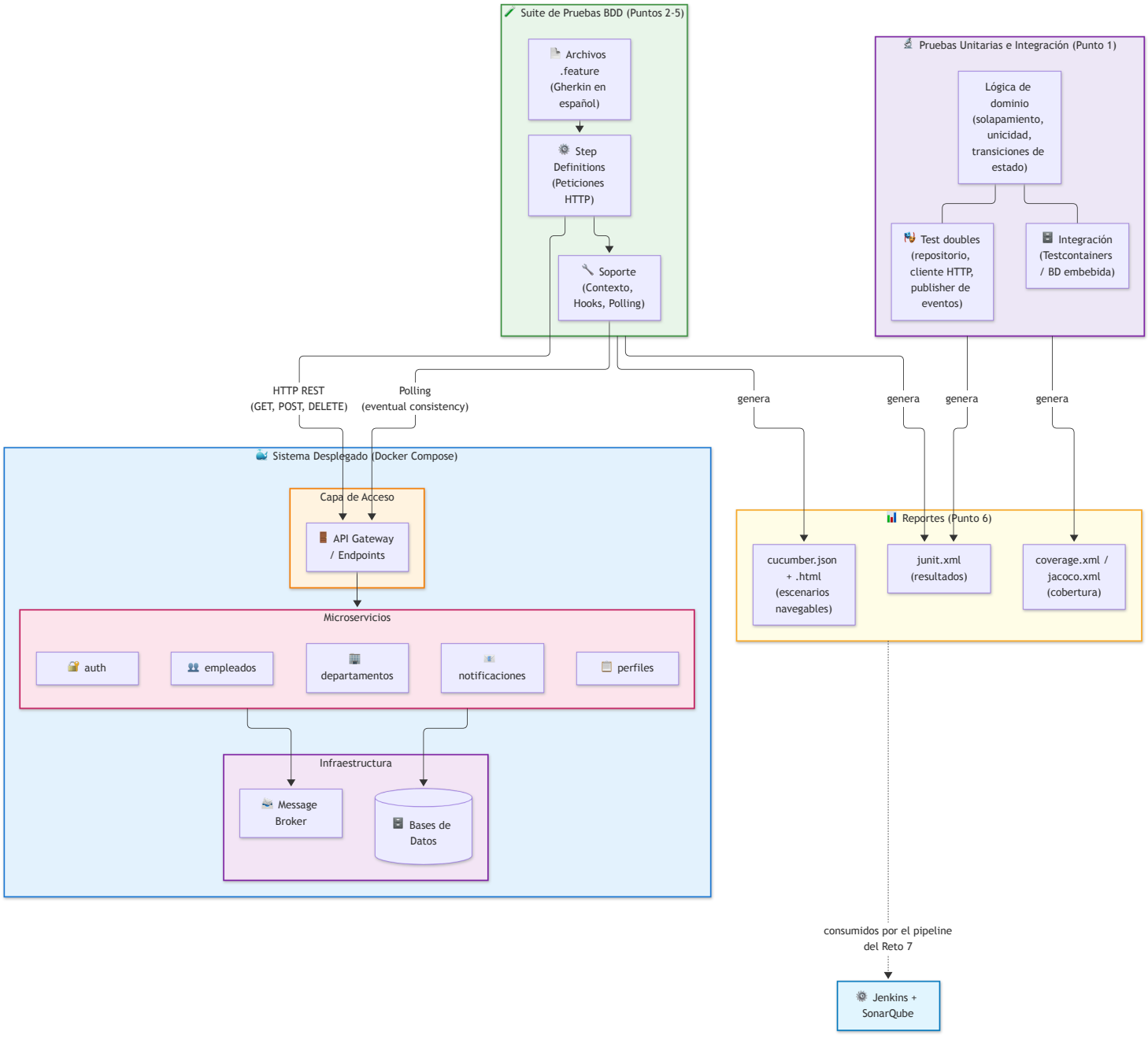
1. **Ejecución con un único comando:** cada suite debe ejecutarse con un solo comando. Como mínimo debe existir:
   - un comando para las pruebas **unitarias y de integración** de cada microservicio (Punto 1),
   - un comando para la suite **BDD** completa, después de levantar el sistema con `docker compose up --build -d`.
2. **Generar los reportes de la suite BDD** en los tres formatos indicados (JUnit XML, Cucumber JSON y HTML), en rutas fijas y documentadas.
3. **Verificar que los reportes son correctos:** abra el HTML y confirme que muestra las características y escenarios con sus pasos; abra el XML y confirme que el conteo de escenarios coincide con lo ejecutado.
4. **Verificar consistencia:** Ejecute la suite completa al menos 3 veces consecutivas y confirme que todos los escenarios pasan de forma consistente. Si hay pruebas intermitentes (*flaky tests*), identifique y corrija la causa.
5. **Simular un fallo:** Modifique un dato esperado en un escenario (ej. cambie un código de respuesta esperado) y verifique que:
   - la prueba falla con un **mensaje descriptivo** que indique qué se esperaba y qué se obtuvo,
   - **el fallo queda registrado en el JUnit XML** con ese mensaje, y el escenario aparece en rojo en el HTML,
   - el comando retorna un **código de salida distinto de cero** (esto es lo que hará fallar el pipeline en el Reto 7).
6. **Tabla de reportes en el README.md.** Documente, para cada suite, dónde queda cada reporte:

Suite	Comando	Reporte de resultados	Reporte de cobertura
`empleados-service` (unit + integración)	`mvn verify`	`target/surefire-reports/*.xml`	`target/site/jacoco/jacoco.xml`
`vacaciones-service` (unit)	`pytest ...`	`reports/junit.xml`	`coverage.xml`
`e2e-tests` (BDD)	`npx cucumber-js ...`	`e2e-tests/reports/junit.xml`	—

   Esta tabla es el **contrato con el Reto 7**: el pipeline de Jenkins va a leer exactamente estas rutas.
7. **Documentar además en el README.md:**
   - Breve explicación de la **pirámide de pruebas** y cómo se distribuyeron las pruebas del equipo entre los tres niveles
   - Breve explicación de qué es BDD y por qué se eligió este enfoque
   - **Prerrequisitos** para ejecutar las pruebas (herramientas, versiones)
   - **Instrucciones exactas** de ejecución paso a paso, incluyendo:
     a. Cómo levantar el sistema
     b. Cómo configurar las variables de entorno
     c. Comando para ejecutar cada suite
     d. Cómo interpretar los resultados y dónde encontrar los reportes
   - **Descripción de los escenarios implementados** (qué flujos se cubren)
   - **Herramientas y frameworks** utilizados con justificación de la elección
8. **(Opcional)** Configurar las pruebas como un **contenedor Docker** dentro del `docker-compose.yml` para que puedan ejecutarse sin instalar dependencias localmente. Si lo hace, **monte el directorio de reportes como volumen**; de lo contrario los reportes desaparecen con el contenedor y Jenkins no podrá publicarlos:

```yaml
services:
  bdd-tests:
    build: ./e2e-tests
    depends_on:
      - empleados-service
      - auth-service
      - notificaciones-service
    environment:
      BASE_URL: http://api-gateway:8080
    volumes:
      - ./e2e-tests/reports:/app/reports   # los reportes deben sobrevivir al contenedor
    networks:
      - microservices-network
```

Diagrama de Arquitectura de Pruebas



Entregables

- **Pruebas unitarias y de integración** en al menos dos microservicios, versionadas junto al código de cada servicio.
- Configuración de generación de **reportes de resultados y de cobertura** en cada microservicio probado.
- Proyecto de pruebas BDD con archivos `.feature` y step definitions, versionado en GitHub.
- Configuración de generación de reportes **JUnit XML, Cucumber JSON y HTML** en la suite BDD.
- README.md actualizado según lo especificado en el Punto 6, incluyendo la **tabla de reportes** (suite → comando → rutas).

Consideraciones

- Las pruebas unitarias **no deben requerir Docker ni el sistema desplegado**. Si una prueba "unitaria" necesita levantar la base de datos, es una prueba de integración.
- Las pruebas de cada microservicio viven **junto al código de ese microservicio**, no en el proyecto `e2e-tests/`.
- Los archivos `.feature` deben estar escritos en **español** usando `# language: es`.
- Las step definitions deben usar **variables de entorno** para configurar URLs base y credenciales.
- Cada escenario debe ser **independiente**: no debe depender del resultado de otro escenario.
- Los pasos que esperan resultados de eventos asíncronos deben implementar **polling con reintentos**, no `sleep` fijo.
- Si un escenario crea datos, use identificadores únicos o limpie el estado entre ejecuciones.
- **Todo comando de pruebas debe retornar un código de salida distinto de cero cuando algo falla.** Un script que siempre retorna `0` hace que el pipeline del Reto 7 pase en verde con pruebas rotas.
- Los directorios de reportes (`reports/` , `target/` , `coverage/`) deben estar en el `.gitignore` : se generan, no se versionan.

Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, distribuidos de la siguiente forma:

#	Reto	Valor	Aspectos a evaluar
1	**Pruebas unitarias, de integración y cobertura**	1.5	Pruebas unitarias de lógica de dominio en al menos **dos microservicios en lenguajes distintos**, incluidas las validaciones de vacaciones (con casos borde del solapamiento) y de empleados. Uso correcto de **test doubles**: ninguna prueba unitaria requiere Docker. Al menos una prueba de **integración** contra base de datos real. Generación de reportes de **resultados** y **cobertura** en formato estándar. **Cobertura ≥ 70 %** en al menos un servicio. Al menos un caso de error probado, no solo el camino feliz.
2	**Configuración del proyecto BDD**	0.5	Proyecto independiente con estructura correcta. Contexto compartido entre pasos bien diseñado. Variables de entorno configuradas. Escenario de humo funcional. Justificación del framework elegido.
3	**Escenarios de seguridad**	0.5	Mínimo 4 escenarios (1 dado + 3 propios). Cubren token ausente, token inválido, diferencia USER vs ADMIN. Steps implementados correctamente. Uso adecuado de `Antecedentes`.
4	**Escenarios de onboarding**	1.0	Escenarios cubren registro, creación de la cuenta vía evento, activación con token, login exitoso, y datos inválidos. Polling implementado correctamente (no `sleep` fijo). Parámetros de polling justificados. Escenarios independientes.
5	**Escenarios de vacaciones y offboarding**	1.0	Escenarios diseñados por el estudiante. Cubren programación de vacaciones, validaciones de rechazo, **ciclo completo de suspensión y reactivación de la cuenta**, desvinculación, bloqueo de login, fallo de recuperación y consulta de auditoría. Precondiciones complejas resueltas. Control del tiempo justificado. Reutilización de pasos.
6	**Reportes, reproducibilidad y documentación**	0.5	Cada suite ejecutable con un comando y **generando sus reportes** (JUnit XML + Cucumber JSON/HTML en la suite BDD). Los comandos retornan código de salida distinto de cero al fallar. Resultados consistentes en 3 ejecuciones. **Tabla de reportes** en el README con las rutas exactas que consumirá el Reto 7.

> **Nota:** Se valorará especialmente la **claridad y expresividad** de los escenarios Gherkin. Un buen escenario debe poder ser entendido por alguien que no conoce el código, actuando como documentación viva del sistema.

> **Nota sobre el Reto 7:** los reportes que se generan aquí son el insumo directo del pipeline de Integración Continua. Un equipo que termine este reto sin reportes en formato estándar quedará bloqueado en el siguiente.